

This is a copy of a conversation between ChatGPT & Anonymous.
Report conversation

現状Google認証でログインすると、下記エラーが出ます。

Access Denied

You do not have permission to sign in.

現在の[...nextauth].jsの中身は下記です。

しかしcallbacksの中身をコメントアウトするとAccess Deniedは解除されます。

```
import NextAuth from "next-auth"
import GoogleProvider from "next-auth/providers/google"
import axios from 'axios'

const apiUrl = process.env.NEXT_PUBLIC_API_URL

export default NextAuth ({
  // Configure one or more authentication providers
  providers: [
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    }),
  ],
  callbacks: {
    async signIn({ user, account, profile }) {
      const provider = account.provider
      const uid = account.sub
      const name = user.name
      const email = user.email

      try {
        const response = await
        axios.post(`${apiUrl}/auth/${provider}/callback
        , {
```

```
        provider,  
        uid,  
        name,  
        email,  
    })  
  
    if (response.status === 200) {  
        return true  
    } else {  
        return false  
    }  
} catch (error) {  
    console.log('エラー', error)  
    return false  
}  
},  
},  
})
```

Google認証で「Access Denied」エラーが発生している原因は、signIn コールバック関数内で何かしらの問題が発生している可能性が高いです。このエラーは、signIn コールバックが false を返すときに発生します。これは、Google認証自体は成功しているが、続く処理（この場合はあなたのAPIへのリクエスト）で問題が発生していることを示しています。

対処法としては、以下のステップで問題の特定と解決を試みることができます。

- 1. エラーの特定:** signIn コールバック内で発生しているエラーを特定します。エラーログを詳細にチェックし、axios.post リクエストが失敗する原因を突き止めます。サーバー側のログも確認してください。
- 2. APIの確認:** `/${apiUrl}/auth/${provider}/callback` へのリクエストが正しく処理されているか確認します。必要に応じて、このAPIのエンドポイントでのエラーハンドリングを強化することを検討してください。
- 3. レスポンスステータスの確認:** `response.status` が200以外の場合に false を返していますが、この条件が適切かどうかを再検討します。例えば、他のステータスコード（例：201 Created）が成功と見なされる場合があります。
- 4. エラーハンドリングの改善:** 現在のエラーハンドリングでは単にコンソールにエラーを出力していますが、エラーの内容に基づいてユーザーに適切なフィードバックを提

供するように改善することを検討してください。

5. **環境変数の確認:** `process.env.GOOGLE_CLIENT_ID`、
`process.env.GOOGLE_CLIENT_SECRET`、および
`process.env.NEXT_PUBLIC_API_URL` が正しく設定されていることを確認してください。
6. **コードのコメントアウト:** 問題解決までの間、`signIn` コールバックをコメントアウトしてアプリケーションの他の部分が正常に動作することを確認することも一つの方法です。

これらのステップを順に実行し、エラーの原因を特定して解決してください。また、問題が解決しない場合は、エラーメッセージやログの詳細を共有していただければ、より具体的なアドバイスが可能です。

ログインしようとするとき下記のようなURLになります。
<http://localhost:8000/api/auth/error?error=AccessDenied>

<http://localhost:8000/api/auth/error?error=AccessDenied> というURLが表示される場合、NextAuthの `signIn` コールバックが `false` を返しており、アクセスが拒否されています。これは、あなたのサーバー側（Ruby on Rails）でユーザーの認証が正しく処理されていない、または `axios.post` リクエストが想定通りに動作していないことを示している可能性があります。

対処法として以下のステップを試してみてください。

1. コールバック関数の検証:

- `signIn` コールバックで `false` が返される条件（`response.status !== 200`）を再確認します。
- `axios.post` のリクエストとレスポンスを詳細にログ出力し、どの段階で問題が発生しているかを特定します。

2. サーバー側のAPIを確認:

- Ruby on Railsの該当APIエンドポイント（`/auth/${provider}/callback`）におけるリクエストの処理方法を確認します。
- サーバー側のログをチェックし、APIが期待通りに動作しているか、または何らかのエラーが発生しているかを確認します。

3. エラーハンドリングの改善:

- エラーが発生した場合のハンドリングを強化し、具体的なエラーメッセージをフロントエンドに返すようにします。
- ユーザーに対してより有用なフィードバックを提供するために、フロントエンドでこれらのエラーメッセージを表示します。

4. 環境変数の再確認:

- .env ファイルや環境設定で NEXT_PUBLIC_API_URL やGoogleのクライアントID、シークレットが正しく設定されているかを再確認します。

5. 簡易テストの実施:

- signIn コールバック内の axios.post を一時的にシンプルなレスポンスを返すコードに置き換え、問題がAPIリクエストに由来するかどうかを確認します。

これらのステップにより問題の原因を特定し、適切な対処を行ってください。さらに詳細なエラーログやコードの断片があれば、より具体的なアドバイスが可能です。

```
Googleで設定している承認済みのリダイレクト URIは下記  
ですが、正しいですか？  
http://localhost:8000/api/auth/callback/google  
apiUrl=http://localhost:8000  
const response = await  
axios.post(`${apiUrl}/api/auth/callback/${provi  
der}`
```

指定されたGoogleの承認済みリダイレクトURI

http://localhost:8000/api/auth/callback/google は、通常、Next.js と NextAuth を使用する際の標準的な設定です。NextAuthはデフォルト

で /api/auth/callback/[provider] のルートを使用してOAuthプロバイダからのコールバックを処理します。したがって、このURLは正しく設定されているように見えます。

ただし、 axios.post で使用しているURLが問題を引き起こしている可能性があります。あなたのコードでは、 axios.post のURLが

`${apiUrl}/api/auth/callback/${provider}` となっています。これは、Googleの承認済みリダイレクトURIと一致していません。

`axios.post` で使用するURLは、NextAuthの `signIn` コールバックからあなたのバックエンド（Ruby on Rails）にリクエストを送信するためのものです。このURLは、Googleの承認済みリダイレクトURIとは異なるものであるべきです。このURLは、ユーザーの認証情報を処理するためのあなたのサーバー側のAPIエンドポイントを指している必要があります。

したがって、次のような手順を検討してください：

- 1. バックエンドのAPIエンドポイントを確認：** Ruby on Railsのバックエンドで、ユーザー認証情報を処理するためのエンドポイントが正しく設定されていることを確認します。このエンドポイントは、Google認証から得られた情報を受け取り、あなたのシステムでユーザーを認証する役割を持ちます。
- 2. `axios`のリクエストURLの確認：** `axios.post` のURLが、上記のバックエンドのAPIエンドポイントを正しく指していることを確認します。このURLは、Googleの承認済みリダイレクトURIとは異なります。
- 3. リダイレクトURIの確認：** Googleの承認済みリダイレクトURIがNextAuthの期待する形式で設定されていることを再確認します。

これらの点を確認し、必要に応じて修正を行ってください。その上で、もしまだ問題が解決しない場合は、具体的なエラーメッセージやログ情報を提供していただければ、さらに詳細なサポートが可能です。

現状ターミナルに下記のエラーが出ています。

```
portfolio_front | エラー AxiosError: connect
ECONNREFUSED 127.0.0.1:8000
portfolio_front |   at AxiosError.from
(file:///front/node_modules/axios/lib/core/AxiosError.js:8
9:14)
portfolio_front |   at
RedirectableRequest.handleRequestError
(file:///front/node_modules/axios/lib/adapters/http.js:606
:25)
portfolio_front |   at RedirectableRequest.emit
(node:events:517:28)
portfolio_front |   at eventHandlers.<computed>
(/front/node_modules/follow-redirects/index.js:14:24)
portfolio_front |   at ClientRequest.emit
```

```
(node:events:517:28)
portfolio_front | at Socket.socketErrorListener
(node:_http_client:501:9)
portfolio_front | at Socket.emit (node:events:517:28)
portfolio_front | at emitErrorNT
(node:internal/streams/destroy:151:8)
portfolio_front | at emitErrorCloseNT
(node:internal/streams/destroy:116:3)
portfolio_front | at process.processTicksAndRejections
(node:internal/process/task_queues:82:21) {
portfolio_front | port: 8000,
portfolio_front | address: '127.0.0.1',
portfolio_front | syscall: 'connect',
portfolio_front | code: 'ECONNREFUSED',
portfolio_front | errno: -111,
portfolio_front | config: {
portfolio_front |   transitional: {
portfolio_front |     silentJSONParsing: true,
portfolio_front |     forcedJSONParsing: true,
portfolio_front |     clarifyTimeoutError: false
portfolio_front |   },
portfolio_front |   adapter: ['xhr', 'http'],
portfolio_front |   transformRequest: [[Function:
transformRequest]],
portfolio_front |   transformResponse: [[Function:
transformResponse]],
portfolio_front |   timeout: 0,
portfolio_front |   xsrfCookieName: 'XSRF-TOKEN',
portfolio_front |   xsrfHeaderName: 'X-XSRF-TOKEN',
portfolio_front |   maxContentLength: -1,
portfolio_front |   maxBodyLength: -1,
portfolio_front |   env: { FormData: [Function], Blob:
[class Blob] },
portfolio_front |   validateStatus: [Function:
validateStatus],
portfolio_front |   headers: Object [AxiosHeaders] {
portfolio_front |     Accept: 'application/json, text/plain,
*/*',
portfolio_front |     'Content-Type': 'application/json',
portfolio_front |     'User-Agent': 'axios/1.6.2',
```

```
portfolio_front | 'Content-Length': '84',
portfolio_front | 'Accept-Encoding': 'gzip, compress,
portfolio_front | deflate, br'
portfolio_front | },
portfolio_front | method: 'post',
portfolio_front | url:
portfolio_front | 'http://localhost:8000/api/auth/callback/google',
portfolio_front | data: '{"provider":"google","name":"オ
portfolio_front | メガマスター","email":"masya868686@gmail.com"}'
portfolio_front | },
portfolio_front | request: <ref *1> Writable {
portfolio_front |   _writableState: WritableState {
portfolio_front |     objectMode: false,
portfolio_front |     highWaterMark: 16384,
portfolio_front |     finalCalled: false,
portfolio_front |     needDrain: false,
portfolio_front |     ending: false,
portfolio_front |     ended: false,
portfolio_front |     finished: false,
portfolio_front |     destroyed: false,
portfolio_front |     decodeStrings: true,
portfolio_front |     defaultEncoding: 'utf8',
portfolio_front |     length: 0,
portfolio_front |     writing: false,
portfolio_front |     corked: 0,
portfolio_front |     sync: true,
portfolio_front |     bufferProcessing: false,
portfolio_front |     onwrite: [Function: bound onwrite],
portfolio_front |     writecb: null,
portfolio_front |     writelen: 0,
portfolio_front |     afterWriteTickInfo: null,
portfolio_front |     buffered: [],
portfolio_front |     bufferedIndex: 0,
portfolio_front |     allBuffers: true,
portfolio_front |     allNoop: true,
portfolio_front |     pendingcb: 0,
portfolio_front |     constructed: true,
portfolio_front |     prefinished: false,
portfolio_front |     errorEmitted: false,
portfolio_front |     emitClose: true,
```

```

portfolio_front |   autoDestroy: true,
portfolio_front |   errored: null,
portfolio_front |   closed: false,
portfolio_front |   closeEmitted: false,
portfolio_front |   [Symbol(kOnFinished)]: []
portfolio_front | },
portfolio_front |   _events: [Object: null prototype] {
portfolio_front |     response: [Function:
handleResponse],
portfolio_front |     error: [Function:
handleRequestError],
portfolio_front |     socket: [Function:
handleRequestSocket]
portfolio_front |   },
portfolio_front |   _eventsCount: 3,
portfolio_front |   _maxListeners: undefined,
portfolio_front |   _options: {
portfolio_front |     maxRedirects: 21,
portfolio_front |     maxBodyLength: Infinity,
portfolio_front |     protocol: 'http:',
portfolio_front |     path: '/api/auth/callback/google',
portfolio_front |     method: 'POST',
portfolio_front |     headers: [Object: null prototype],
portfolio_front |     agents: [Object],
portfolio_front |     auth: undefined,
portfolio_front |     family: undefined,
portfolio_front |     beforeRedirect: [Function:
dispatchBeforeRedirect],
portfolio_front |     beforeRedirects: [Object],
portfolio_front |     hostname: 'localhost',
portfolio_front |     port: '8000',
portfolio_front |     agent: undefined,
portfolio_front |     nativeProtocols: [Object],
portfolio_front |     pathname:
'/api/auth/callback/google'
portfolio_front |   },
portfolio_front |   _ended: false,
portfolio_front |   _ending: true,
portfolio_front |   _redirectCount: 0,
portfolio_front |   _redirects: [],

```



```

portfolio_front | _requestBodyLength: 84,
portfolio_front | _requestBodyBuffers: [ [Object] ],
portfolio_front | _onNativeResponse: [Function
(anonymous)],
portfolio_front | _currentRequest: ClientRequest {
portfolio_front |   _events: [Object: null prototype],
portfolio_front |   _eventsCount: 7,
portfolio_front |   _maxListeners: undefined,
portfolio_front |   outputData: [],
portfolio_front |   outputSize: 0,
portfolio_front |   writable: true,
portfolio_front |   destroyed: false,
portfolio_front |   _last: true,
portfolio_front |   chunkedEncoding: false,
portfolio_front |   shouldKeepAlive: false,
portfolio_front |   maxRequestsOnConnectionReached:
false,
portfolio_front |   _defaultKeepAlive: true,
portfolio_front |   useChunkedEncodingByDefault: true,
portfolio_front |   sendDate: false,
portfolio_front |   _removedConnection: false,
portfolio_front |   _removedContLen: false,
portfolio_front |   _removedTE: false,
portfolio_front |   strictContentLength: false,
portfolio_front |   _contentLength: '84',
portfolio_front |   _hasBody: true,
portfolio_front |   _trailer: '',
portfolio_front |   finished: false,
portfolio_front |   _headerSent: true,
portfolio_front |   _closed: false,
portfolio_front |   socket: [Socket],
portfolio_front |   _header: 'POST
/api/auth/callback/google HTTP/1.1\r\n' +
portfolio_front |     'Accept: application/json, text/plain,
*/*\r\n' +
portfolio_front |     'Content-Type: application/json\r\n'
+
portfolio_front |     'User-Agent: axios/1.6.2\r\n' +
portfolio_front |     'Content-Length: 84\r\n' +
portfolio_front |     'Accept-Encoding: gzip, compress,

```

```

deflate, br\r\n' +
portfolio_front | 'Host: localhost:8000\r\n' +
portfolio_front | 'Connection: close\r\n' +
portfolio_front | '\r\n',
portfolio_front | _keepAliveTimeout: 0,
portfolio_front | _onPendingData: [Function: nop],
portfolio_front | agent: [Agent],
portfolio_front | socketPath: undefined,
portfolio_front | method: 'POST',
portfolio_front | maxHeaderSize: undefined,
portfolio_front | insecureHTTPParser: undefined,
portfolio_front | joinDuplicateHeaders: undefined,
portfolio_front | path: '/api/auth/callback/google',
portfolio_front | _ended: false,
portfolio_front | res: null,
portfolio_front | aborted: false,
portfolio_front | timeoutCb: null,
portfolio_front | upgradeOrConnect: false,
portfolio_front | parser: null,
portfolio_front | maxHeadersCount: null,
portfolio_front | reusedSocket: false,
portfolio_front | host: 'localhost',
portfolio_front | protocol: 'http:',
portfolio_front | _redirectable: [Circular *1],
portfolio_front | [Symbol(kCapture)]: false,
portfolio_front | [Symbol(kBytesWritten)]: 0,
portfolio_front | [Symbol(kNeedDrain)]: false,
portfolio_front | [Symbol(corked)]: 0,
portfolio_front | [Symbol(kOutHeaders)]: [Object: null
prototype],
portfolio_front | [Symbol(error)]: null,
portfolio_front | [Symbol(kHighWaterMark)]: 16384,
portfolio_front |
[Symbol(kRejectNonStandardBodyWrites)]: false,
portfolio_front | [Symbol(kUniqueHeaders)]: null
portfolio_front | },
portfolio_front | _currentUrl:
'http://localhost:8000/api/auth/callback/google',
portfolio_front | [Symbol(kCapture)]: false
portfolio_front | },

```

```
portfolio_front | cause: Error: connect ECONNREFUSED
127.0.0.1:8000
portfolio_front |   at TCPConnectWrap.afterConnect [as
oncomplete] (node:net:1555:16) {
portfolio_front |   errno: -111,
portfolio_front |   code: 'ECONNREFUSED',
portfolio_front |   syscall: 'connect',
portfolio_front |   address: '127.0.0.1',
portfolio_front |   port: 8000
portfolio_front | }
portfolio_front | }
```

ECONNREFUSED 127.0.0.1:8000 のエラーは、`axios.post` リクエストがローカルホストのポート8000に接続できなかったことを示しています。このエラーは通常、リクエスト先のサーバーが動作していないか、指定されたポートでリッスンしていない場合に発生します。

この問題を解決するために、以下の点を確認してみてください。

1. Ruby on Railsサーバーの稼働確認:

- Ruby on Railsのサーバーが正しく起動していることを確認します。ターミナルで `rails server` または `rails s` コマンドを実行して、サーバーが起動しているか確認してください。

2. ポート番号の確認:

- Railsサーバーがポート8000で動作していることを確認します。デフォルトでは、Railsは通常ポート3000で起動します。もしポート番号が異なる場合は、`axios.post` のURLに使用されているポート番号を適切に修正する必要があります。

3. ネットワーク設定の確認:

- Dockerを使用している場合、ネットワーク設定に問題がないかを確認します。Dockerコンテナ内からホストマシンのローカルネットワークにアクセスするには、`localhost` ではなく、特定のDockerのネットワーク設定を使用する必要があります。

4. ファイアウォールやセキュリティ設定の確認:

- システムのファイアウォールやセキュリティソフトウェアがポート8000の通信をブロックしていないか確認してください。

5. ローカルホストアドレスの確認:

- ローカル開発環境であるため、127.0.0.1 (localhost) が正しいアドレスであることを確認します。場合によっては、Dockerのネットワーク設定によっては、異なるアドレスを使用する必要があるかもしれません。

これらの確認を行い、問題が解決しない場合は、さらに詳細な環境設定やエラーログを提供いただければ、具体的なアドバイスを行うことができます。

ネットワーク設定の確認に関してですが、dockerを使用しています。

Dockerを使用している場合、ネットワーク設定が ECONNREFUSED エラーの原因である可能性が高いです。Dockerコンテナ内からホストマシンにアクセスするためには、通常の localhost や 127.0.0.1 ではなく、特定のアドレスを使用する必要があります。

以下の手順で問題を解決できるか確認してください：

1. ホストマシンへのアクセス:

- Dockerコンテナ内からホストマシンにアクセスするには、localhost の代わりに host.docker.internal を使用します。したがって、axios.post のURLを `http://host.docker.internal:8000/api/auth/callback/google` のように修正してみてください。

2. Docker Composeのネットワーク設定:

- docker-compose.yml ファイルで、異なるサービス（例えばフロントエンドとバックエンド）が同じネットワーク上にあることを確認します。これにより、サービス間で通信が容易になります。

3. ポートフォワーディングの確認:

- Dockerコンテナが使用するポート（この場合は8000）が、ホストマシンのポートにフォワーディングされていることを確認します。docker-compose.yml ファイルにて、ports セクションで適切なポートフォワーディングが設定されているかを確認してください。

4. サービスの依存関係の設定:

- docker-compose.yml で、フロントエンドサービスがバックエンドサービスに依存していることを明示的に指定することもできます。これは depends_on オプションを使用して行います。

フロントエンドの確認

バックエンドの確認

るか確認します。コンテナが予期せず終了していないか、または何らかのエラーが発生していないかをチェックします。

これらのステップを通じて、ネットワークの問題が解決するか確認してみてください。それでも問題が解決しない場合は、Dockerの設定ファイルや具体的なエラーメッセージを提供いただければ、さらに詳細なアドバイスが可能です。

ターミナルで下記のエラーが出ています。

```
portfolio_front | [next-auth][error]
[OAUTH_CALLBACK_ERROR]
portfolio_front | https://next-
auth.js.org/errors#oauth_callback_error State cookie was
missing. {
portfolio_front |   error: TypeError: State cookie was
missing.
portfolio_front |   at Object.use
(/front/node_modules/next-
auth/core/lib/oauth/checks.js:111:23)
portfolio_front |   at OAuthCallback
(/front/node_modules/next-
auth/core/lib/oauth/callback.js:89:25)
portfolio_front |   at
process.processTicksAndRejections
(node:internal/process/task_queues:95:5)
portfolio_front |   at async Object.callback
(/front/node_modules/next-
auth/core/routes/callback.js:52:11)
portfolio_front |   at async AuthHandler
(/front/node_modules/next-auth/core/index.js:302:28)
portfolio_front |   at async NextAuthApiHandler
(/front/node_modules/next-auth/next/index.js:22:19)
portfolio_front |   at async NextAuth._args$
(/front/node_modules/next-auth/next/index.js:108:14) {
portfolio_front |   name: 'OAuthCallbackError',
portfolio_front |   code: undefined
portfolio_front | },
portfolio_front |   providerId: 'google',
```

```
portfolio_front | message: 'State cookie was missing.'  
portfolio_front |}
```

このエラー「State cookie was missing.」は、NextAuth.jsのOAuthコールバック処理中に発生しており、OAuth認証プロセスにおいて「state」クッキーが見つからないことを示しています。この「state」クッキーは、CSRF攻撃を防ぐためにOAuthプロトコルで重要な役割を果たします。

解決策を以下に示します。

1. クッキー設定の確認:

- NextAuth.jsがクッキーを正しく設定しているかを確認します。特に、`secure` 属性や `SameSite` 属性が適切に設定されているかが重要です。これらの属性は、HTTPS接続や特定のサイト間リクエストのコンテキストでクッキーの動作を制御します。

2. HTTPSの使用:

- 開発環境がHTTP（非セキュア）ではなくHTTPS（セキュア）を使用していることを確認します。多くのOAuthプロバイダは、セキュアな接続（HTTPS）上でのみクッキーを正しく処理します。

3. ブラウザの設定を確認:

- ブラウザがクッキーを適切に扱っているか確認します。ブラウザのプライバシー設定やセキュリティ設定が厳しすぎると、必要なクッキーがブロックされることがあります。

4. NextAuth.jsの設定を確認:

- NextAuth.jsの設定を見直し、特に `session` や `jwt` の設定が適切であることを確認します。これらの設定は認証プロセスにおいて重要な役割を果たします。

5. フロントエンドとバックエンド間のCORS設定:

- CORS（Cross-Origin Resource Sharing）設定が適切に行われているか確認します。不適切なCORS設定は、ブラウザが異なるオリジン間でクッキーを適切に扱うのを妨げる可能性があります。

6. サーバーログの確認:

- OAuthプロバイダ（この場合はGoogle）とNextAuth.jsサーバー間のコミュニケーションに関するログを確認し、追加のエラー情報を探します。

これらのチェックを行い、問題が解決しない場合は、NextAuth.jsのバージョンを確認し、必要に応じて最新バージョンにアップデートすることも検討してください。また、

NextAuth.jsのドキュメントやコミュニティでサポートを求めるのも良い方法です。